

Overview Materi Pelatihan

Implementing Docker for Web Developer

Dokumen ini merangkum inti materi dari setiap bab modul pelatihan Implementing Docker for Web Developer, mulai dari pengenalan arsitektur Docker, bahasa dan pustaka pemrograman web, pembuatan serta distribusi image, jaringan Docker, pemilihan distribusi sistem operasi yang optimal, hingga studi kasus penerapan aplikasi. Bagian akhir dokumen memuat rangkaian Lab praktik yang disusun mengikuti studi kasus peserta pelatihan.

Bab 1 – Introduction and Docker Architecture

Bab ini mengenalkan konsep dasar Docker: apa itu Docker, cara kerjanya, serta arsitektur dan komponen yang membentuk ekosistemnya.

- Definisi Docker dan cara kerjanya (client – daemon – registry)
- Komponen arsitektur: Docker Daemon, Docker Client, Docker Host, Docker Registry, dan Docker Image
- Komponen Docker Engine: dockerd (server), REST API, dan CLI
- Fitur utama: Docker Engine, Docker Hub, Docker Compose, serta Docker Desktop untuk Mac/Linux/Windows
- Manfaat Docker bagi development, konfigurasi, code pipeline, debugging, multitenancy, dan penskalaan resource
- Siklus hidup container: created → running → paused/stopped → deleted
- Gambaran umum Docker Networking & Storage
- Kelebihan Docker: konfigurasi sederhana, keamanan baik, mendukung multi-cloud, mudah di-debug, dan mendukung multi-OS

Bab 2 – Web Apps Programming Language and Library

Bab ini membahas berbagai bahasa pemrograman dan pustaka (library) yang umum digunakan dalam pengembangan aplikasi web berbasis Docker.

- Perbedaan bahasa pemrograman sisi frontend dan backend
- Bahasa pemrograman populer: JavaScript, Python, TypeScript, PHP, Ruby, serta HTML & CSS
- Pustaka pemrograman pendukung: NumPy, Matplotlib, TensorFlow, dan Apache Spark

Bab 3 – Image Creation and Sharing

Bab ini menjelaskan konsep Docker Image, struktur penyusunnya, serta cara pembuatan dan distribusinya melalui registry.

- Definisi Docker Image dan alasan pembuatannya
- Anatomi image: image layer, container layer, parent image, base image, dan docker manifest
- Jenis container registry: Docker Hub, registry pihak ketiga, dan self-hosted registry
- Proses pembuatan image: Dockerfile → build → tar → deploy

Bab 4 – Docker Networking

Bab ini membahas cara container Docker saling terhubung, baik dalam satu host maupun antar host, serta ke jaringan luar (internet).

- Definisi dan tujuan Docker Networking, termasuk Container Network Model (CNM) Driver
- Empat tipe network bawaan Docker: Bridge, None, Host, dan Mac Vlan
- Konsep Libnetwork serta komponen CNM: Network Controller, Driver, Network, Endpoint, dan Sandbox
- Karakteristik Docker Network Drivers (native driver vs remote/third-party driver, IPAM)
- Model jaringan container yang umum digunakan: Host Network, Bridge Network, dan Overlay Network (swarm mode)

Bab 5 – Optimized Operating System Distributions for Docker

Bab ini mengulas perbandingan Virtual Machine dan Container, serta beberapa distribusi sistem operasi yang dioptimalkan untuk menjalankan Docker.

- Perbandingan isolasi Virtual Machine vs Container
- Manfaat memilih distribusi OS yang tepat: peningkatan kinerja & efisiensi, penggunaan resource lebih rendah, dan peningkatan keamanan
- Distribusi OS yang diulas: Alpine Linux, Fedora CoreOS, RancherOS (discontinued), Ubuntu Core, dan Flatcar Container Linux
- Kesimpulan: pemilihan distribusi OS disesuaikan dengan kebutuhan penggunaan dan beban kerja yang spesifik

Bab 6 – Application Use Cases

Bab penutup ini membahas dasar-dasar pengujian aplikasi Docker serta diakhiri dengan sesi praktik studi kasus.

- Dasar-dasar pengujian (objektivitas, prinsip dasar testing, dan testabilitas)
- Gambaran umum siklus hidup pengujian (test lifecycle)
- Integrasi pengujian dalam siklus hidup pengembangan software (SDLC)
- Sesi praktik studi kasus (Praktik Studi Case)

Studi Kasus Pelatihan: Pengelolaan Website CMS

Seluruh sesi lab pada modul ini disusun mengikuti studi kasus nyata yang diajukan peserta pelatihan, yaitu pengelolaan website berbasis CMS (WordPress/Joomla) di lingkungan kerja masing-masing. Studi kasus ini mencakup empat fokus utama yang dipraktikkan secara bertahap pada Lab 1 sampai Lab 6:

- Migrasi website dari server VM ke Docker (Joomla/WordPress)
- Troubleshooting error website (502, database, permission)
- Analisis dan pembersihan malware/backdoor pada website
- Optimasi dan pengamanan website (hardening & konfigurasi server, termasuk enrichment server Docker development)

Setiap lab pada bagian berikut menggunakan image resmi versi terbaru dari Docker Hub agar peserta terbiasa dengan rilis dan praktik terkini. Karena tag versi image dapat diperbarui secara berkala, disarankan mengecek ulang versi terbaru di Docker Hub sesaat sebelum sesi berlangsung.

Lab 1 – Instalasi Docker dan Eksplorasi Arsitektur

Tujuan: memasang Docker Engine versi terbaru dan mempraktikkan komponen arsitektur (client, daemon, registry) dari Bab 1, sebagai fondasi sebelum memigrasikan website CMS pada Lab 3.

Image/tools yang digunakan:

- Docker Engine terbaru (instalasi via skrip resmi get.docker.com atau Docker Desktop versi terkini)
- hello-world:latest
- alpine:latest untuk eksplorasi container dasar

Langkah kerja:

- 1 Pasang Docker Engine terbaru sesuai OS host (Linux/Windows/Mac) mengikuti dokumentasi resmi Docker.
- 2 Verifikasi instalasi dengan `docker --version` dan `docker info`.
- 3 Jalankan `docker run hello-world` untuk memastikan client dapat berkomunikasi dengan daemon dan menarik image dari registry (Docker Hub).
- 4 Jalankan `docker run -it alpine:latest sh` untuk masuk ke dalam container dan amati siklus hidup container dengan `docker ps -a`.
- 5 Bersihkan container percobaan dengan `docker rm` dan image yang tidak terpakai dengan `docker image prune`.

Kaitan studi kasus: pastikan versi Docker Engine di server tujuan migrasi sudah versi stabil terbaru sebelum melanjutkan ke Lab 3.

Lab 2 – Menjalankan Aplikasi Web PHP dalam Container

Tujuan: mempraktikkan menjalankan aplikasi berbasis PHP di dalam container menggunakan image bahasa pemrograman resmi terbaru (Bab 2), sebagai dasar sebelum mengelola CMS berbasis PHP (WordPress/Joomla) pada Lab 3.

Image/tools yang digunakan:

- php:8.3-apache (image resmi PHP terbaru dengan Apache bawaan)
- node:22-alpine (opsional, untuk simulasi build tool front-end modern)

Langkah kerja:

- 6 Buat berkas index.php sederhana yang menampilkan phpinfo().
- 7 Jalankan `docker run -d -p 8080:80 -v $(pwd):/var/www/html php:8.3-apache` dan akses melalui browser di localhost:8080.
- 8 Amati bagaimana container memetakan port dan volume, sebagai persiapan konsep yang sama saat men-deploy WordPress/Joomla.
- 9 Hentikan container dan diskusikan perbedaan varian image -apache dan -fpm yang akan dipakai bersama Nginx pada Lab 4.

Lab 3 – Migrasi Website CMS dari VM ke Docker (WordPress/Joomla)

Tujuan: memigrasikan website CMS (WordPress atau Joomla) yang sebelumnya berjalan di server VM ke dalam container Docker, mempraktikkan pembuatan image dan registry (Bab 3), sekaligus menjawab fokus studi kasus #1 (migrasi VM ke Docker).

Image/tools yang digunakan:

- wordpress:6-php8.3-apache (image resmi WordPress terbaru) atau joomla:5-php8.3-apache (image resmi Joomla terbaru)
- mariadb:11 atau mysql:8.4 sebagai database
- Docker Compose V2 (bawaan Docker Engine terbaru)
- Registry tujuan: Docker Hub (private repository) atau self-hosted registry (misalnya Harbor) sesuai kebijakan internal

Langkah kerja:

- 10 Backup data dari server VM: `mysqldump` basis data CMS dan arsipkan folder konten (wp-content untuk WordPress, atau images/media untuk Joomla).

- 11 Susun docker-compose.yml dengan service db (mariadb:11/mysql:8.4) dan service cms (wordpress:6-php8.3-apache/joomla:5-php8.3-apache), lengkap dengan volume persisten untuk data dan konten.
- 12 Jalankan docker compose up -d, lalu impor dump basis data ke service db menggunakan docker exec -i <container_db> mysql ...
- 13 Salin folder konten hasil backup VM ke volume container CMS menggunakan docker cp atau mount volume langsung.
- 14 Jika dibutuhkan plugin/ekstensi khusus, buat Dockerfile turunan (FROM wordpress:6-php8.3-apache atau FROM joomla:5-php8.3-apache) untuk menambahkan modul PHP/berkas tambahan, lalu build dengan docker build -t <namespace>/cms-web:latest .
- 15 Beri tag versi image sesuai kebutuhan (docker tag) dan unggah ke registry tujuan dengan docker push.
- 16 Verifikasi website dapat diakses, konfigurasi domain/URL pada CMS sudah sesuai lingkungan baru, dan uji fungsi utama (login admin, upload media, tampilan halaman).